CPS3233 - Verification Techniques - ModelJUnit Tutorial

Resources

ModelJUnit source code including Javadocs:
https://sourceforge.net/p/modeljunit/code/HEAD/tree/trunk/modeljunit/

Information on how to structure your test class:
http://www.cse.chalmers.se/edu/year/2015/course/DAT260/labs/modeljunit-tutorial.html

Process

- Analyse the system by reading the question well. Treat the system as a black box. You should only be interested in the interface and not in the specific implementation of each method. You job is to create a driver which interacts with the public methods.
- Create a list of the states and transitions. Additionally, create a list of what variables you need to track (state variables). There is no need to look at the implementation to create a model.
- Use the states and transitions you listed to create a visual model.
- Write down which state variables are updated when a specific transition happens.
- This concludes most of the 'paper work.' It is now time to start coding.
- Create your enum state class.
- Create a model class which implements the FSM model and create an instance of the system under test.
- Define your starting state and create your state variables.
- Implement the necessary methods.
    - getState() should give you the state of the model at any given time.
    - reset(boolean) should reset the model to the initial values. If *true* is passed as a parameter then the SUT is also updated to the original state. This is called randomly by the system itself.
- Implement the transitions.
    - Each transition must have a guard which checks whether the transition can be performed. This guard checks the current state and any other necessary conditions. The guard's name should be the same as the transition with the 'Guard' suffix.
    - The transition itself (implemented using the annotation @Action) has to do three things
        - Update the system under test by executing the relevant method(s).
        - Update the model's state variables.
        - Check correspondence between the system under test and the model via assertions. Each transition might need to assert multiple things. Only test the variables being changed.
- Implement the test runner. Decide which type of tester you want to use (Random Tester? Greedy Tester? etc.) along with the coverage metrics.
- You can either chose to generate a number of predefined transitions or have the model run for a specified period of time. Remember, transition choices are random so don't generate 8 transitions if your model has 8 transitions. Use a number which will consistently cover all transitions.
- If your model fails, then you have a problem with the model implementation (assuming the system's implementation is correct). Recheck the guard's conditions and whether you are correctly updating the model in comparison to the system.

NOTE: This is not a waterfall process. You will have to go back to the drawing board and recreate the paper model many times before you get it right. My suggestion would be to adopt a rapid prototyping approach where you create a model, try to implement it, modify the model, implement again etc.